Monoid & Semigroup

son: 6 · FP Foundations · Category: Functional · Level: Intermediate · Build with Wa

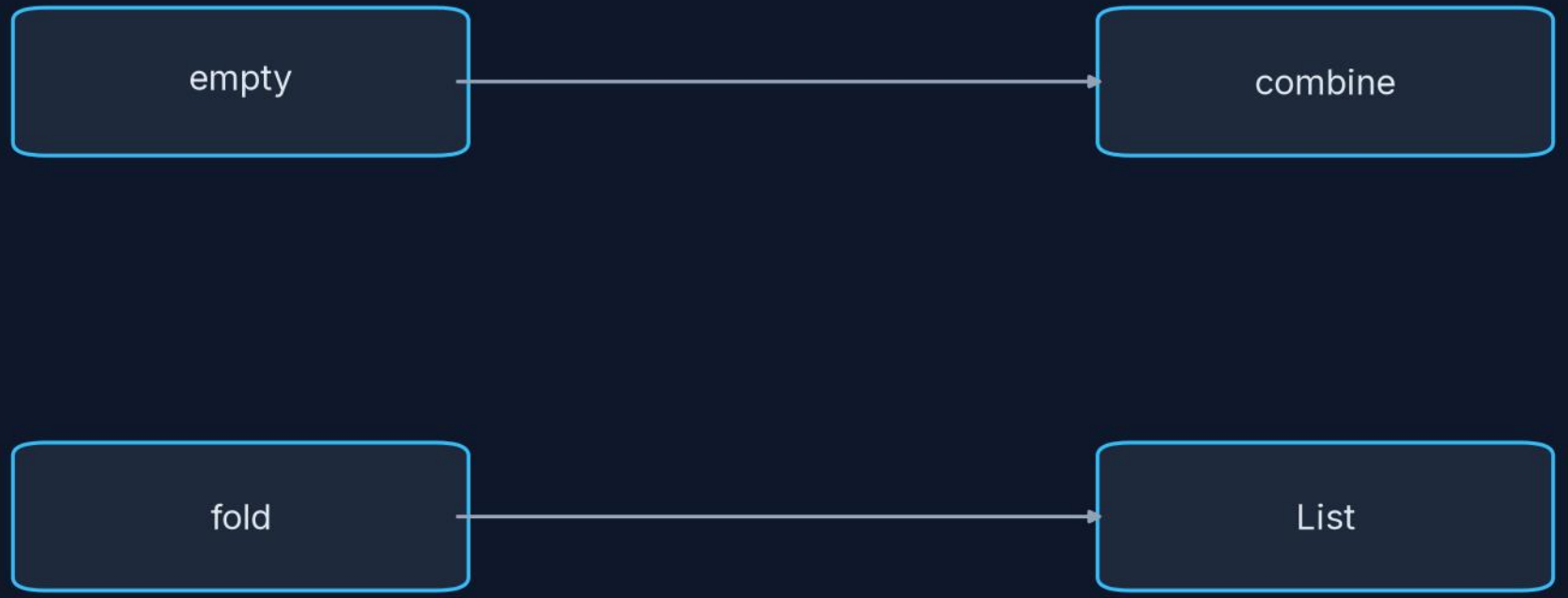
Today's Hook

- Today: Monoid & Semigroup as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Monoid & Semigroup structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Monoid & Semigroup
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/monoid/Monoids.java
package com.buildwithwaqar.patterns.functional.monoid;

public final class Monoids {
    private Monoids() {}

    public static final Monoid<Integer> INT_SUM = new Monoid<>() {
        public Integer empty() { return 0; }
        public Integer combine(Integer a, Integer b) { return a + b; }
    };

    public static final Monoid<String> STRING = new Monoid<>() {
        public String empty() { return ""; }
        public String combine(String a, String b) { return a + b; }
    };
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.monoid;
import java.util.List;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Monoid walkthrough ===");
    System.out.println("sum=" + Monoids.INT_SUM.fold(List.of(1,2,3,4)));
    System.out.println("str=" + Monoids.STRING.fold(List.of("Build"," ","with"," ","Waqar")));
  }
}
```

Walkthrough Output

```
=== Monoid walkthrough ===
sum=10
str=Build with Waqar
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Monoid & Semigroup
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Monoid & Semigroup → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.